



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №10
«ВВЕДЕНИЕ В МАШИННОЕ
ОБУЧЕНИЕ»**

по дисциплине «Языки программирования для статистической обработки
данных»

Студент группы

ИНБО-20-23. Боргачев Т.М.

(подпись)

Преподаватель

Трушин СМ

(подпись)

Москва 2025 г.

ВВЕДЕНИЕ

Цель работы: изучить основы машинного обучения на данных о связи музыкальных предпочтений и психического здоровья. Построить модели классификации (Music effects) и регрессии (Anxiety, Depression).

Задачи:

1. Построить модели логистической регрессии (классификация) и линейной регрессии (регрессия).
2. Оценить качество моделей с помощью метрик (Accuracy, MSE).
3. Проанализировать ошибки через матрицы ошибок и графики остатков.
4. Сравнить результаты в Python и R.

Данные:

- Датасет: **mxmh_survey_results.csv**
- Признаки: Возраст, часы прослушивания, жанры, тревожность, депрессия и др.
- Целевые переменные:
 - Классификация: **Music effects** (Improve/No effect/Worsen).
 - Регрессия: **Anxiety, Depression**.

РЕЗУЛЬТАТЫ РАБОТЫ

Подготовка данных

Код для предварительной подготовки данных на Python представлен в листинге 1.

Листинг 1 – Предобработка данных в Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Загрузка данных
df = pd.read_csv("mxmh_survey_results.csv")
df = df.drop(columns=['Timestamp'])
# Кодирование категориальных признаков
cat_columns = df.select_dtypes(include=['object']).columns

# Применяем LabelEncoder к каждому категориальному столбцу
label_encoder = LabelEncoder()
for col in cat_columns:
    df[col] = label_encoder.fit_transform(df[col].astype(str))

numeric_columns = df.select_dtypes(include=['float64', 'int64']).columns
# Заполняем пропуски в числовых столбцах средним значением
for col in numeric_columns:
    df[col] = df[col].fillna(df[col].mean())

# Разделение на train/test для классификации
X_class = df.drop(['Music effects', 'Anxiety', 'Depression'], axis=1)
y_class = df['Music effects']
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(
    X_class, y_class, test_size=0.3, random_state=42
)

# Разделение на train/test для регрессии
X_reg = df.drop(['Anxiety', 'Depression'], axis=1)
y_reg = df['Anxiety']
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.3, random_state=42
)
```

Код для предварительной подготовки данных на R представлен в листинге 2.

Листинг 2 – Предобработка данных в R

```
library(caret)
library(nnet)

# Чтение данных
df <- read.csv("mxmh_survey_results.csv")

# Удаление столбца Timestamp
if ("Timestamp" %in% colnames(df)) {
  df$Timestamp <- NULL
}

# Кодирование категориальных признаков
cat_columns <- names(df)[sapply(df, is.character)]
for (col in cat_columns) {
  # Преобразуем в фактор с сохранением порядка уровней
  df[[col]] <- factor(df[[col]])
  # Кодировем в числовой формат
  df[[col]] <- as.numeric(df[[col]])
}

# Заполнение пропусков в числовых столбцах
numeric_columns <- names(df)[sapply(df, is.numeric)]
for (col in numeric_columns) {
  df[[col]][is.na(df[[col]])] <- mean(df[[col]], na.rm = TRUE)
}

# Преобразуем целевую переменную в фактор
df$Music.effects <- as.factor(df$Music.effects)

# Подготовка данных для классификации
X_class <- df[, !names(df) %in% c("Music.effects", "Anxiety", "Depression")]
y_class <- df$Music.effects
temp_class <- cbind(X_class, Music.effects = y_class)

# Разделение на train/test для классификации
set.seed(42)
trainIndex_class <- createDataPartition(temp_class$Music.effects, p = 0.7, list =
FALSE)
train_class <- temp_class[trainIndex_class, ]
test_class <- temp_class[-trainIndex_class, ]

# Разделение на train/test для регрессии
trainIndex_reg <- createDataPartition(df$Anxiety, p=0.7, list=FALSE)
train_reg <- df[trainIndex_reg, ]
test_reg <- df[-trainIndex_reg, ]
```

Валидация моделей и оценка качества

Код для модели классификации на Python представлен в листинге 3.

Листинг 3 – Логистическая регрессия в Python

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Обучение модели
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_class, y_train_class)
y_pred_class = logreg.predict(X_test_class)

# Метрики
acc = accuracy_score(y_test_class, y_pred_class)
cm = confusion_matrix(y_test_class, y_pred_class)
print(f"Accuracy: {acc:.2f}")
print("Confusion Matrix:\n", cm)
```

Код для модели классификации на R представлен в листинге 4.

Листинг 4 – Логистическая регрессия в R

```
# Обучение модели
model_log <- multinom(Music.effects ~ ., data = train_class)

# Прогнозирование на тестовых данных
pred_class <- predict(model_log, newdata = test_class)

# Преобразуем предсказания и тестовые данные в факторы с одинаковыми уровнями
levels_test <- levels(test_class$Music.effects)
pred_class <- factor(pred_class, levels = levels_test)
actual_class <- factor(test_class$Music.effects, levels = levels_test)

# Оценка точности модели
conf_matrix <- confusionMatrix(pred_class, actual_class)
accuracy <- conf_matrix$overall["Accuracy"]
print(paste("Accuracy:", round(accuracy, 2)))
```

Код для модели регрессии на Python представлен в листинге 5.

Листинг 5 – Линейная регрессия в Python

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Обучение модели
linreg = LinearRegression()
linreg.fit(X_train_reg, y_train_reg)
y_pred_reg = linreg.predict(X_test_reg)

# Метрики
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f"MSE: {mse:.2f}")
```

Код для модели регрессии на R представлен в листинге 6.

Листинг 6 – Линейная регрессия в R

```
model_lin <- lm(Anxiety ~ ., data=train_reg)
pred_reg <- predict(model_lin, newdata=test_reg)

# Метрики
mse <- mean((test_reg$Anxiety - pred_reg)^2)
print(paste("MSE:", round(mse, 2)))
```

Сравнение результатов

Полученные метрики для регрессии и классификации на Python представлены на рис. 1.

```
Метрики классификации:
Accuracy: 0.76
Confusion Matrix:
[[166   1   0]
 [ 51   2   0]
 [  1   0   0]]
Метрики регрессии:
MSE: 7.24
R2_score: 0.07
MAE: 2.22
```

Рисунок 1 – Метрики, полученные в Python

Полученные метрики для регрессии и классификации на R представлены на рис. 2.

```

[1] "Метрики классификации: "
[1] "Accuracy: 0.73"
Confusion Matrix and Statistics

      Reference
Prediction  1    2    3    4
      1     0     1     7     0
      2     2    155    39     4
      3     0     5     4     1
      4     0     1     0     0

Overall Statistics

      Accuracy : 0.726
      95% CI : (0.6619, 0.784)
      No Information Rate : 0.7397
      P-Value [Acc > NIR] : 0.708

      Карра : 0.1263

      McNemar's Test P-Value : NA

Statistics by Class:

                Class: 1 Class: 2 Class: 3 Class: 4
Sensitivity      0.000000   0.9568   0.08000 0.000000
Specificity      0.963134   0.2105   0.96450 0.995327
Pos Pred Value   0.000000   0.7750   0.40000 0.000000
Neg Pred Value   0.990521   0.6316   0.77990 0.977064
Prevalence       0.009132   0.7397   0.22831 0.022831
Detection Rate   0.000000   0.7078   0.01826 0.000000
Detection Prevalence 0.036530   0.9132   0.04566 0.004566
Balanced Accuracy 0.481567   0.5837   0.52225 0.497664
[1] "Метрики регрессии: "
[1] "MSE: 5.14"
[1] "MAE: 1.76"
[1] "R-squared (R2): 0.32"

```

Рисунок 2 – Метрики, полученные в R

1. Метрики классификации:

- Accuracy: в Python точность модели составляет 0.76, в то время как в R она немного ниже — 0.73. Это может быть связано с различиями в реализации алгоритмов или настройками моделей.
- Confusion Matrix: матрица ошибок в Python показывает, что модель лучше справляется с классификацией первого класса (166 верных предсказаний), но хуже работает с остальными классами. В R же модель демонстрирует более равномерное распределение ошибок, но при этом значительно хуже предсказывает первый класс (0 верных предсказаний). Это может указывать на проблемы с балансировкой классов или недостаточную сложность модели.
- Карра: в R Карра равен 0.1263, что указывает на низкое согласие между предсказаниями и истинными значениями. Это подтверждает, что модель плохо обобщает данные для нескольких классов.

2. Метрики регрессии:

- MSE (Mean Squared Error): в Python MSE равно 7.24, а в R — 5.14. Более низкое значение MSE в R говорит о том, что модель в R делает

предсказания, которые в среднем ближе к реальным значениям.

- MAE (Mean Absolute Error): аналогично, MAE в Python составляет 2.22, а в R — 1.76. Это также подтверждает, что модель в R работает лучше по абсолютной ошибке.
- R^2 (R-squared): значение R^2 в Python равно 0.07, что указывает на очень слабую объяснительную способность модели. В R R^2 выше (0.32), но всё равно остаётся относительно низким. Это говорит о том, что даже улучшенная модель в R не может полностью объяснить зависимость в данных.

Возможные причины различий и ограничения линейной регрессии

1. Сложность зависимости в данных:

- Результаты метрик (R^2 , MSE, MAE) указывают на то, что зависимость между признаками и целевой переменной, вероятно, является нелинейной. Линейная регрессия предполагает, что связь между переменными может быть описана линейной функцией, что часто не соответствует реальности. Если в данных присутствуют сложные взаимодействия (например, полиномиальные зависимости, нелинейные тренды или выбросы), линейная модель не сможет их адекватно описать.

2. Различия в реализации:

- Разница в результатах между Python и R может быть вызвана различиями в способах предобработки данных, нормализации признаков или использовании гиперпараметров. Например, если в одной из реализаций применялась стандартизация данных, это могло повлиять на качество модели.

3. Проблемы с данными:

- Низкие значения R^2 в обеих реализациях могут быть связаны с шумом в данных, мультиколлинеарностью признаков или недостаточным количеством релевантных признаков. Это особенно заметно в случае Python, где R^2 близко к нулю.

ЗАКЛЮЧЕНИЕ

- Сильные стороны Python:
 - Удобные библиотеки (sklearn) для быстрого прототипирования.
 - Гибкость в настройке моделей.
- Сильные стороны R:
 - Встроенные статистические отчеты (например, `summary(model)`).
 - Удобство визуализации с `ggplot2`.
- Рекомендации:
 - Для задач с большими данными использовать Python.
 - Для академических исследований — R.